

Informe de reflexión - Parte II

Pruebas del pipeline CI/CD y métricas DORA propias

Integrantes: José Vanegas y Miguel Vanegas
Repositorio: github.com/migu145/inventario-app
Fecha: 27 de julio de 2026

1. Verificación reproducible

Para ordenar la entrega fuimos colocando cada comando en el README junto con la salida que obtuvimos. Las capturas originales están en evidencias/capturas-reales. También dejamos los dos registros de despliegue y el archivo CSV que usamos para calcular DORA. Así se puede repetir la práctica siguiendo el mismo orden.

Elemento comprobado	Evidencia del repositorio	Resultado observado
Pipeline base	Secciones README 8 a 17; capturas 52 a 60	6 pruebas, build multi-stage, Actions en verde, GHCR y Rolling Update 2/2
Blue-Green	Capturas 65 y 66; blue-green-final.txt	Service en slot=green; respuesta v2/green; rollback a Blue verificado
Buenas prácticas	Capturas 59, 66, 67 y 68	Secret sin exponer la clave, Trivy sin CRITICAL y readiness 503 hasta quedar 1/1

2. Justificación de Blue-Green

Escogimos Blue-Green porque era la forma más directa de mostrar el cambio entre dos versiones. El endpoint /version indica la versión, el color y el pod que respondió. Primero probamos Green sin mover el tráfico y después cambiamos el selector slot del Service. Blue quedó activo por si era necesario regresar.

Pensamos también en Canary, pero para esta práctica pequeña habría sido más difícil demostrar qué porcentaje de solicitudes llegaba a cada versión. Blue-Green usa más recursos porque mantiene los dos ambientes, aunque el cambio y el rollback se entienden mejor al revisar las salidas de los comandos.

3. Persistencia al recrear el pod

En esta prueba agregamos un producto desde la interfaz y luego eliminamos el pod que lo había guardado. El dato estaba en /app/data/products.json, dentro de ese contenedor. Cuando Kubernetes creó el reemplazo, el producto ya no apareció. También notamos que cada réplica tenía su propia copia del catálogo.

Conclusión de la prueba: recrear el pod recupera la aplicación, pero no los cambios hechos en sus archivos. Para conservar el catálogo entre réplicas necesitaríamos una base de datos o un volumen compartido.

4. Métricas DORA con datos propios

Tomamos la hora del commit desde Git y la comparamos con la hora en que kubectl rollout status confirmó el cambio en el clúster. También anotamos la finalización de cada run de GitHub Actions. Todas las horas están en UTC-05.

Lead time for changes. Los dos tiempos fueron 00:03:31 y 00:02:19. El promedio es 00:02:55, que corresponde al nivel Élite de la tabla usada.

Frecuencia de despliegue. Promovimos dos cambios correctos durante un día de trabajo. El resultado es 2 despliegues por día, también clasificado como Élite.

Change failure rate. Registramos tres intentos en total. Uno necesitó una corrección posterior por ImagePullBackOff. El cálculo es $1 / 3 \times 100$, por lo que el resultado es 33,33 % y el nivel es Medio.

#	SHA	Commit	Run correcto	En el clúster	Lead time
1	8a26dc3	25-jul 16:57:29	Run 30176640875 16:59:15	25-jul 17:01:00	00:03:31
2	85eaa0f	25-jul 17:02:01	Run 30176786798 17:03:10	25-jul 17:04:20	00:02:19

Fuentes: Git, runs 30176640875 y 30176786798, evidencias/despliegue-8a26dc3.txt, evidencias/despliegue-85eaa0f.txt y evidencias/dora-deployments.csv.

5. Problemas reales y resolución

BuildKit y las pruebas: al principio la etapa final no dependía de la etapa test, por lo que el build podía terminar sin ejecutar la suite. Cambiamos el Dockerfile para que producción copie el código desde test.

Imagen incorrecta: una etiqueta SHA que no existía produjo ImagePullBackOff. Revisamos GHCR, usamos una etiqueta publicada y repetimos el rollout hasta tener 2/2 pods.

Readiness: /health respondía 200 incluso durante el arranque lento. Agregamos STARTUP_DELAY_SECONDS y una prueba que comprueba primero el 503 y después el 200.

Trivy y PowerShell: Trivy agotó el tiempo al descargar su base y PowerShell modificaba el JSON de kubectl patch. Usamos el repositorio alternativo de Trivy y guardamos los parches en archivos JSON.

6. Reflexión final

Lo que más trabajo nos tomó fue comprobar que la versión correcta realmente estaba corriendo. Una salida parecía suficiente, pero otra prueba mostraba un problema, como ocurrió con la etiqueta inexistente y con /health durante el arranque. También entendimos por qué el catálogo no debe guardarse dentro del contenedor. Si repitiéramos la práctica, validaríamos el SHA en GHCR antes de actualizar el Deployment; eso habría evitado el intento fallido que elevó el CFR.